

Message Board

A major design philosophy of Django, which you will see repeatedly in this book, is that it allows for customization. For example, we don't have to use port 8000 for local development. We could change it to any other available port, such as 8080. Stop the server with `Ctrl + c` and restart it by passing in the desired port number.

Shell

```
(.venv) $ python manage.py runserver 8080
```

If you refresh `http://127.0.0.1:8000/`, you will see an error message, but switching to `http://127.0.0.1:8080/` displays our Hello, World! greeting.

A database is a place to store and access different types of data, and there are two main types of databases: **relational** and **non-relational**.

A relational database stores information in tables containing columns and rows, roughly analogous to an Excel spreadsheet. The columns define what information can be stored; the rows contain the actual data.

Frequently, data in separate tables have some relation to each other, hence the term “relational database” to describe databases with this structure of tables, columns, and rows.

SQL (Structured Query Language) is typically used to interact with relational databases to perform basic CRUD (Create, Read, Update, Delete) operations and define the type of relationship (like a many-to-one relationship, for example).

Django's ORM

An ORM (Object-Relational Mapper) is a powerful programming technique that makes working with data and relational databases much easier. In the case of Django, its ORM means we can write Python code to define database models; we don't have to write raw SQL ourselves. And we don't have to worry about subtle differences in how each database interprets SQL.

Instead, the Django ORM supports five relational databases: SQLite, PostgreSQL, MySQL, MariaDB, and Oracle.

It also comes with support for migrations, which provides a way to track and sync database changes over time.

The Django ORM saves developers a tremendous amount of time, which is one of the major reasons why Django is so efficient.

We will have only one table called “Post” and a single field, “text,” containing the contents of a message.

The actual database table with rows of actual messages would look like this:

Post Database Table

Post

TEXT

My first message board post.

A 2nd post!

A third message.

Django imports a module, `models`, to help us build new database models that will “model” the characteristics of the data in our database

Code

```
# posts/models.py
from django.db import models

class Post(models.Model): # new
    text = models.TextField()
```

Activating Models

After creating a model, the next step is activating it. From now on, whenever we make or modify an existing model, we'll need to update Django in a two-step process:

1. First, we create a migrations file with the `makemigrations` command. Migration files record any changes to the database models, which means we can track changes over time and debug errors as necessary.
2. Second, we build the database with the `migrate` command, which executes the instructions in our migrations file.

Ensure the local server is stopped by typing `Control+c` on the command line and then run the commands `python manage.py makemigrations posts` and `python manage.py migrate`.

Shell

```
(.venv) $ python manage.py makemigrations posts
```

```
Migrations for 'posts':
```

```
posts/migrations/0001_initial.py
```

```
- Create model Post
```

```
(.venv) $ python manage.py migrate
```

```
Operations to perform:
```

```
Apply all migrations: admin, auth, contenttypes, posts, sessions
```

```
Running migrations:
```

```
Applying posts.0001_initial... OK
```

Django Admin

One of Django's killer features is its robust admin interface that visually interacts with data.

To use the Django admin, we must first create a superuser who can log in. In your command line console, type

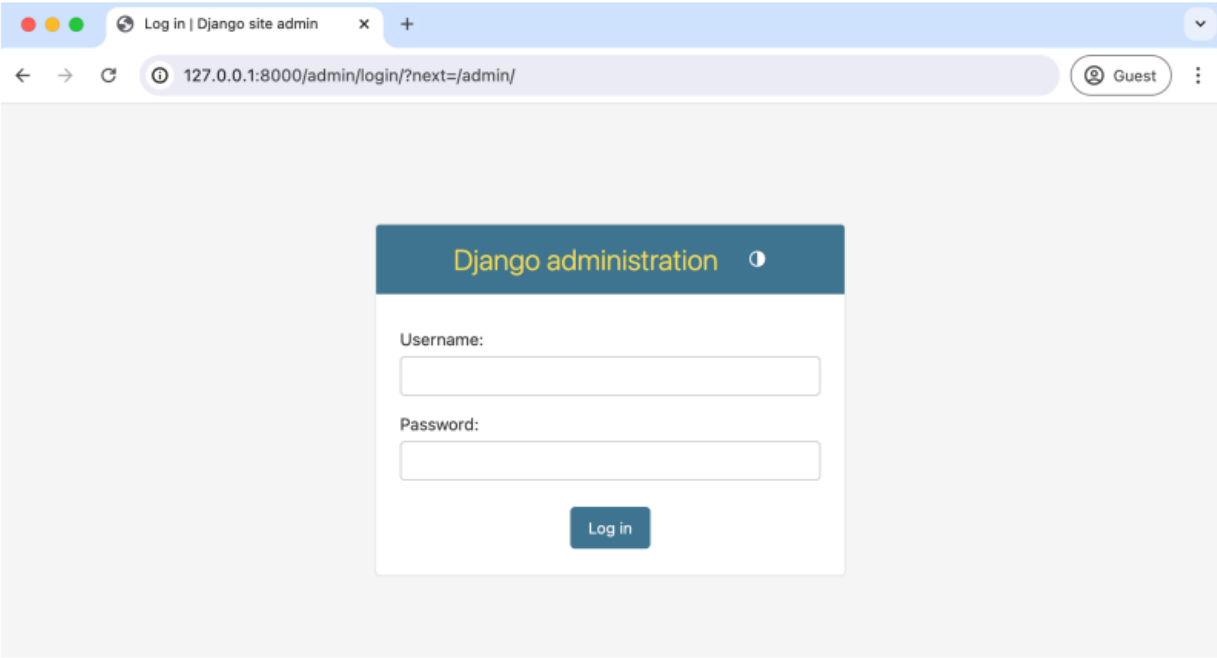
```
python manage.py createsuperuser
```

and respond to the prompts for a username, email, and password:

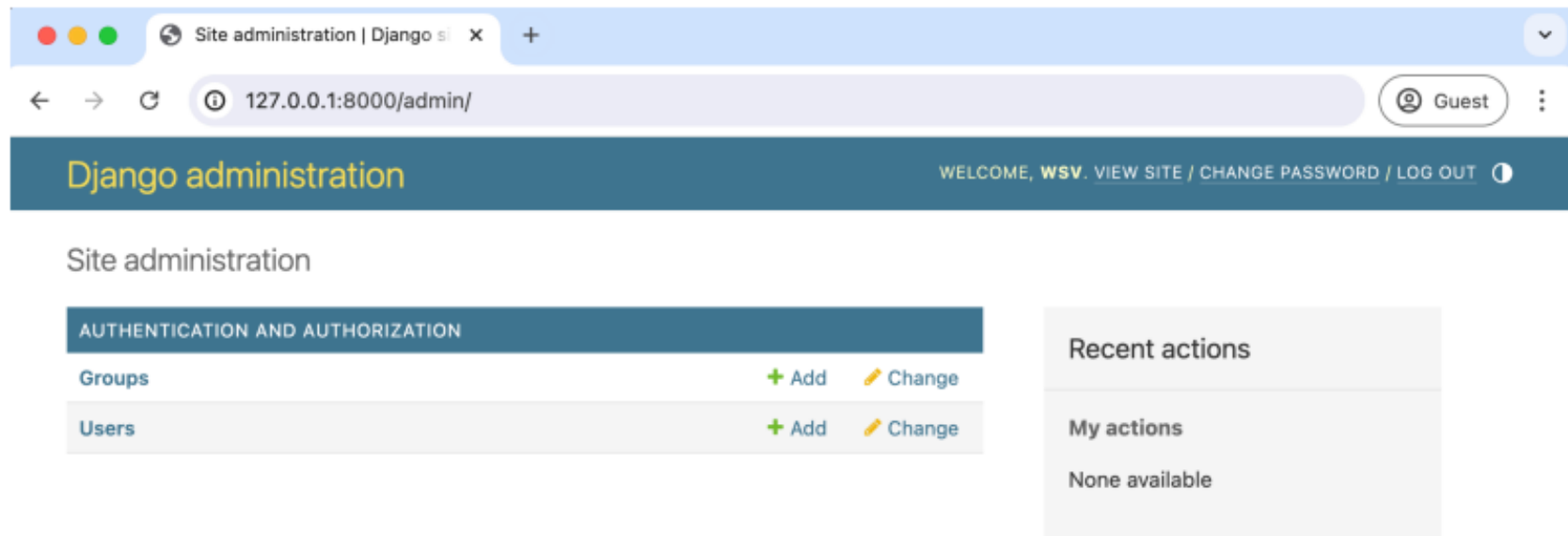
Shell

```
(.venv) $ python manage.py createsuperuser  
Username (leave blank to use 'wsv'): wsv  
Email: will@learndjango.com  
Password:  
Password (again):  
Superuser created successfully.
```

When you type your password, it will not appear visible in the command line console for security reasons. For local development, I often use `testpass123`. Restart the Django server with `python manage.py runserver` and, in your web browser, go to `http://127.0.0.1:8000/admin/`. You should see the login screen for the admin:

A screenshot of a web browser window showing the Django administration login page. The browser's address bar displays the URL `127.0.0.1:8000/admin/login/?next=/admin/`. The page features a central white login box with a dark blue header that reads "Django administration". Inside the box, there are two input fields: "Username:" and "Password:". Below these fields is a dark blue "Log in" button. The background of the page is a light gray.

Admin Login Page



Admin Homepage

But where is our posts app since it is not displayed on the main admin page? Just as we must explicitly add new apps to the `INSTALLED_APPS` config, we must also update an app's `admin.py` file for it to appear in the admin.

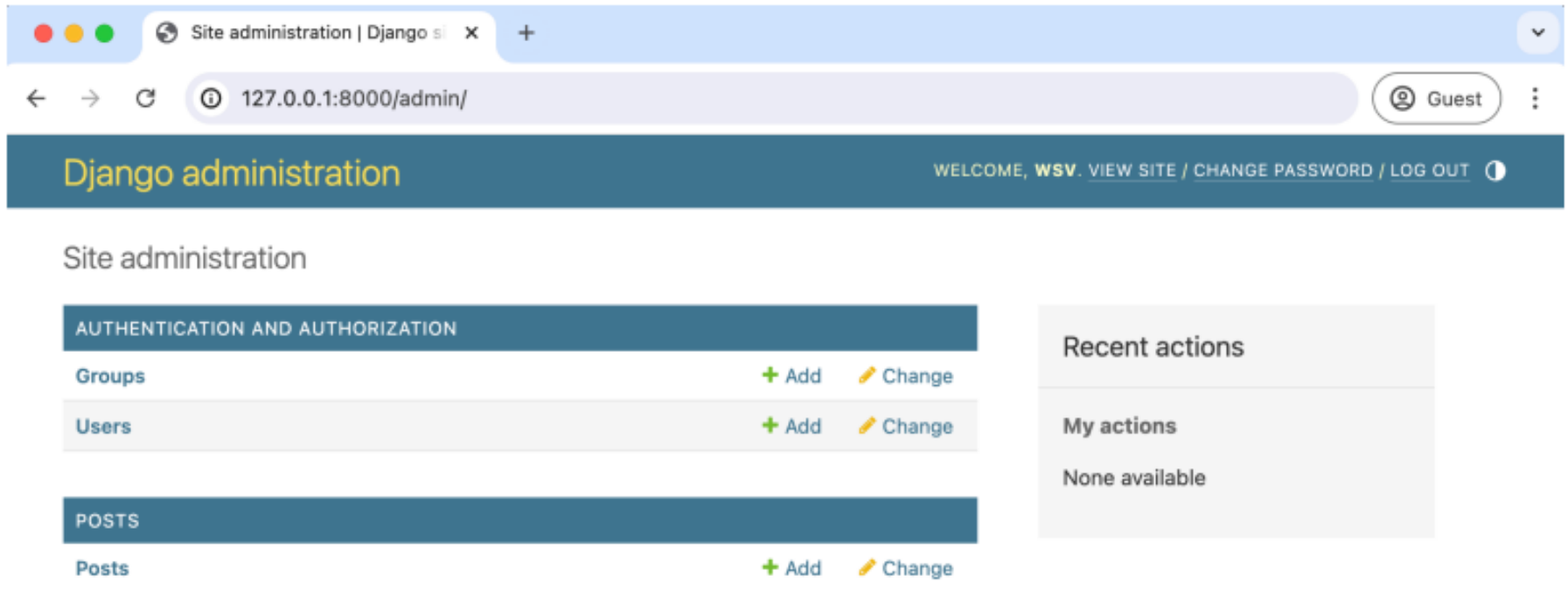
In your text editor, open up `posts/admin.py` and add the following code to display the Post model.

Code

```
# posts/admin.py
from django.contrib import admin

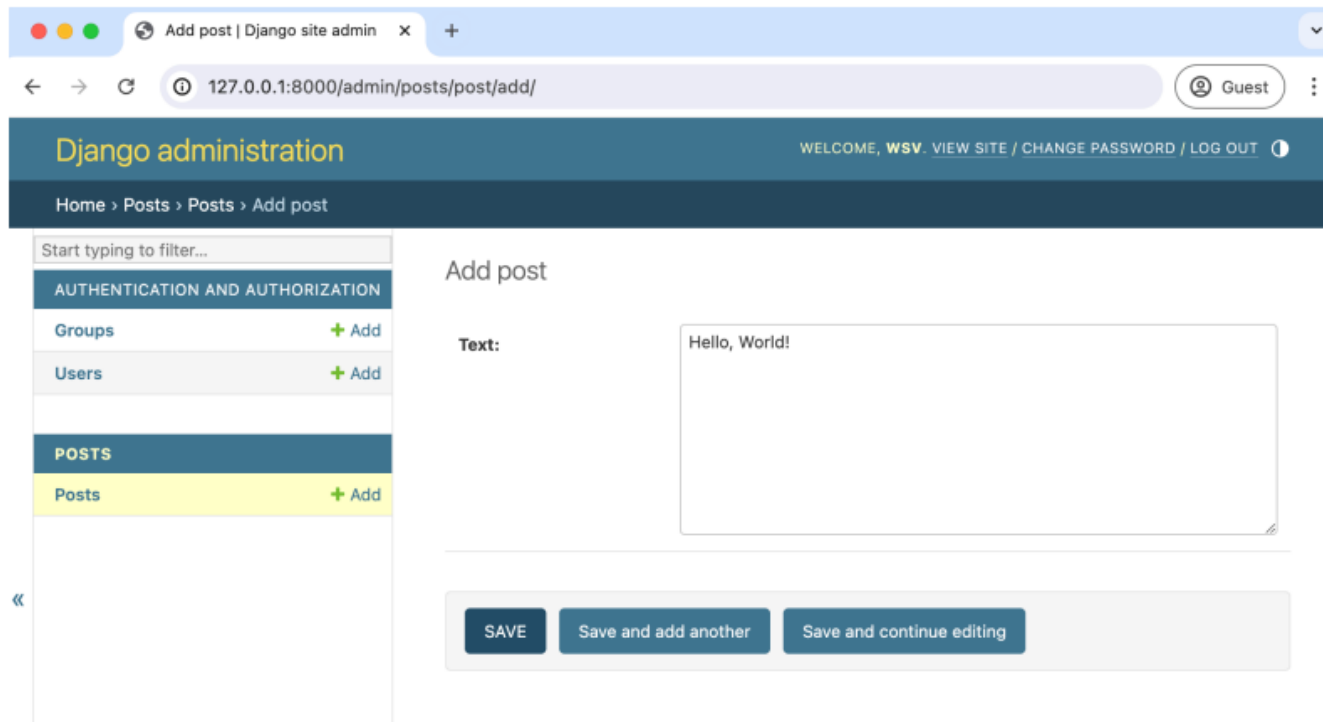
from .models import Post

admin.site.register(Post)
```



Admin Updated Homepage

Let's create our first message board post for our database. Click the + Add button opposite Posts and enter your content in the Text form field.



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:8000/admin/posts/post/add/'. The page title is 'Add post | Django site admin'. The Django administration interface is visible, with a header bar showing 'Django administration' and a welcome message for 'WSV'. The breadcrumb trail is 'Home > Posts > Posts > Add post'. On the left sidebar, the 'POSTS' section is expanded, and the 'Posts' link is highlighted with a '+ Add' button. The main content area is titled 'Add post' and contains a 'Text:' label next to a large text input field. The input field contains the text 'Hello, World!'. At the bottom of the form, there are three buttons: 'SAVE', 'Save and add another', and 'Save and continue editing'.

Admin New Entry

Then click the “Save” button to redirect you to the main Post page. However, if you look closely, there's a problem: our new entry is called “Post object (1)”, which isn't very descriptive!

Select post to change | Django

127.0.0.1:8000/admin/posts/post/

Guest

Django administration

WELCOME, WSV. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Posts > Posts

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

POSTS

Posts + Add

The post "Post object (1)" was added successfully.

Select post to change

ADD POST +

Action:

 Go 0 of 1 selected

☐ POST

☐ Post object (1)

1 post

Admin Posts List

Let's change that. Within the `posts/models.py` file, add a new method called `__str__`, which provides a [human-readable representation](#)⁸⁸ of the model. In this case, we'll have it display the first 50 characters of the text field.

Code

```
# posts/models.py
from django.db import models

class Post(models.Model):
    text = models.TextField()

    def __str__(self): # new
        return self.text[:50]
```

Select post to change | Django

127.0.0.1:8000/admin/posts/post/

Guest

Django administration

WELCOME, **WSV** · [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Posts > Posts

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

POSTS

Posts + Add

Select post to change

ADD POST +

Action:

 Go 0 of 1 selected

☐ POST

☐ Hello, World!

1 post

Select post to change | Django

127.0.0.1:8000/admin/posts/post/

Guest

Django administration

WELCOME, WSV. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Posts > Posts

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups + Add

Users + Add

POSTS

Posts + Add

✓ The post "And this is the 3rd post." was added successfully.

Select post to change

ADD POST +

Action:

 Go 0 of 3 selected

☐ POST

☐ And this is the 3rd post.

☐ This is my second post.

☐ Hello, World!

3 posts

Admin with Three Posts

To display our message board posts on the homepage we have to wire up a view, template, and URL. This pattern should start to feel familiar now.

On the first line, we import the `render()` shortcut function, which combines a template with a context dictionary and returns an `HttpResponse` object. Then, we import our database model, `Post`, from the `models.py` file.

Code

```
# posts/views.py
from django.shortcuts import render
from .models import Post

def post_list(request):
    posts = Post.objects.all()
    return render(request, "post_list.html", {"posts": posts})
```

Let's examine `Post.objects.all()` in more detail as it is the first example of a Database query via the Django ORM.

1. `Post`: This refers to our model class, which represents a table in the database where each row corresponds to an instance of the `Post` model
2. `objects`: This is the default manager for the `Post` model. A [manager](#)⁹⁰ provides a way to interact with the database and perform queries. By default, Django adds a Manager named “objects” to every Django model class.
3. `all()`: This is a method provided by the manager that returns a `QuerySet` containing all instances of the `Post` model from the database. A [QuerySet](#)⁹¹ is a collection of database queries to retrieve objects.

Code

```
<!-- templates/post_list.html -->
<h1>Message Board Homepage</h1>
<ul>
  {% for post in posts %}
    <li>{{ post.text }}</li>
  {% endfor %}
</ul>
```

Code

```
# django_project/urls.py
from django.contrib import admin
from django.urls import path, include # new

urlpatterns = [
    path("admin/", admin.site.urls),
    path("", include("posts.urls")), # new
]
```

Then, in your text editor, create a new `urls.py` file within the `posts` app and update it like so:

Code

```
# posts/urls.py
from django.urls import path
from .views import post_list

urlpatterns = [
    path("", post_list, name="post_list"),
]
```



Message Board Homepage

- Hello, World!
- This is my second post.
- And this is the 3rd post.

Homepage with Three Posts

Listing out all items in a database model is so common that a generic class-based view exists for this, too, called ListView.

Code

```
# posts/views.py
# from django.shortcuts import render
# from .models import Post

# def post_list(request):
#     posts = Post.objects.all()
#     return render(request, "post_list.html", {"posts": posts})

from django.views.generic import ListView # new
from .models import Post

class PostList(ListView): # new
    model = Post
    template_name = "post_list.html"
```

By default, ListView returns a context variable called `<model>_list`, where `<model>` is our model name. Since our model is named `post`, we need to loop over `post_list` with our for loop. The rest of the template remains the same.

Code

```
<!-- templates/post_list.html -->
<h1>Message board homepage</h1>
<ul>
    {% for post in post_list %}
    <li>{{ post.text }}</li>
    {% endfor %}
</ul>
```

```
# posts/tests.py
from django.test import TestCase
from django.urls import reverse # new

from .models import Post

class PostTests(TestCase):
    @classmethod
    def setUpTestData(cls):
        cls.post = Post.objects.create(text="This is a test!")

    def test_model_content(self):
        self.assertEqual(self.post.text, "This is a test!")

    def test_url_exists_at_correct_location(self): # new
        response = self.client.get("/")
        self.assertEqual(response.status_code, 200)

    def test_url_available_by_name(self): # new
        response = self.client.get(reverse("home"))
        self.assertEqual(response.status_code, 200)

    def test_template_name_correct(self): # new
        response = self.client.get(reverse("home"))
        self.assertTemplateUsed(response, "post_list.html")
```